

Luca Sepe

Platform Architecture Reviews

Kubernetes APIs, controllers, control planes, and Go infrastructure

Italy / Remote First

- luca.sepe@gmail.com
- linkedin.com/in/lucasepe
- github.com/lucasepe
- leanpub.com/u/lucasepe

I provide fixed scope, primarily asynchronous architecture reviews for platform teams. I examine one difficult component or decision, trace the available evidence, and return explicit risks, trade offs, and prioritized next actions.

When I am useful

- A Kubernetes API may be difficult to evolve safely.
- A controller works, but its failure semantics remain unclear.
- A control plane has capable components but weak ownership boundaries.
- A proposal needs an external challenge before implementation expands.
- A technical handover leaves important knowledge implicit.

Representative Engineering Work

- Designed and implemented core components of a Kubernetes native platform.
- Built JSON Schema to CRD conversion tooling.
- Implemented reusable controller behavior with dynamic clients and unstructured resources.
- Developed custom Kubernetes API servers and infrastructure software in Go.
- Published practical technical books on controllers, production Go, SRE workflows, and context for AI tools.

Engagement model

- One agreed component or consequential design decision.
- Written intake and repository, code, or document review.
- Primarily asynchronous work with an optional short opening call.
- Written assessment with prioritized findings and next actions.
- One structured handover call; no ongoing staff augmentation.

Expected outputs

- Confirmed behavior separated from inference and unknowns.
- Risks connected to practical technical consequences.
- API, ownership, reconciliation, and failure assumptions made explicit.
- Decisions and trade offs the team can discuss concretely.
- A prioritized path the internal team can own.

Technical Author

Kubernetes Context Engineering

Operational context for humans, SRE workflows, tools, and AI agents.

Go Apps on Kubernetes

Production practices for Go services on Kubernetes.

Backpressure in Microservices

Bounded Queues, Worker Pools, Cancellation, Drop Policies, and Overload Control in Go.

Working principles

- Inspect reality before declaring authority.
- Keep architecture grounded in implementation.
- Turn important constraints into checks.
- Use the smallest sufficient authoritative context.
- Leave the internal team able to own the result.

Send one review question, the repository or material available, and the decision it must support. I will confirm whether it fits a fixed scope review and which inputs are required.